

FIT2099 Assignment 2

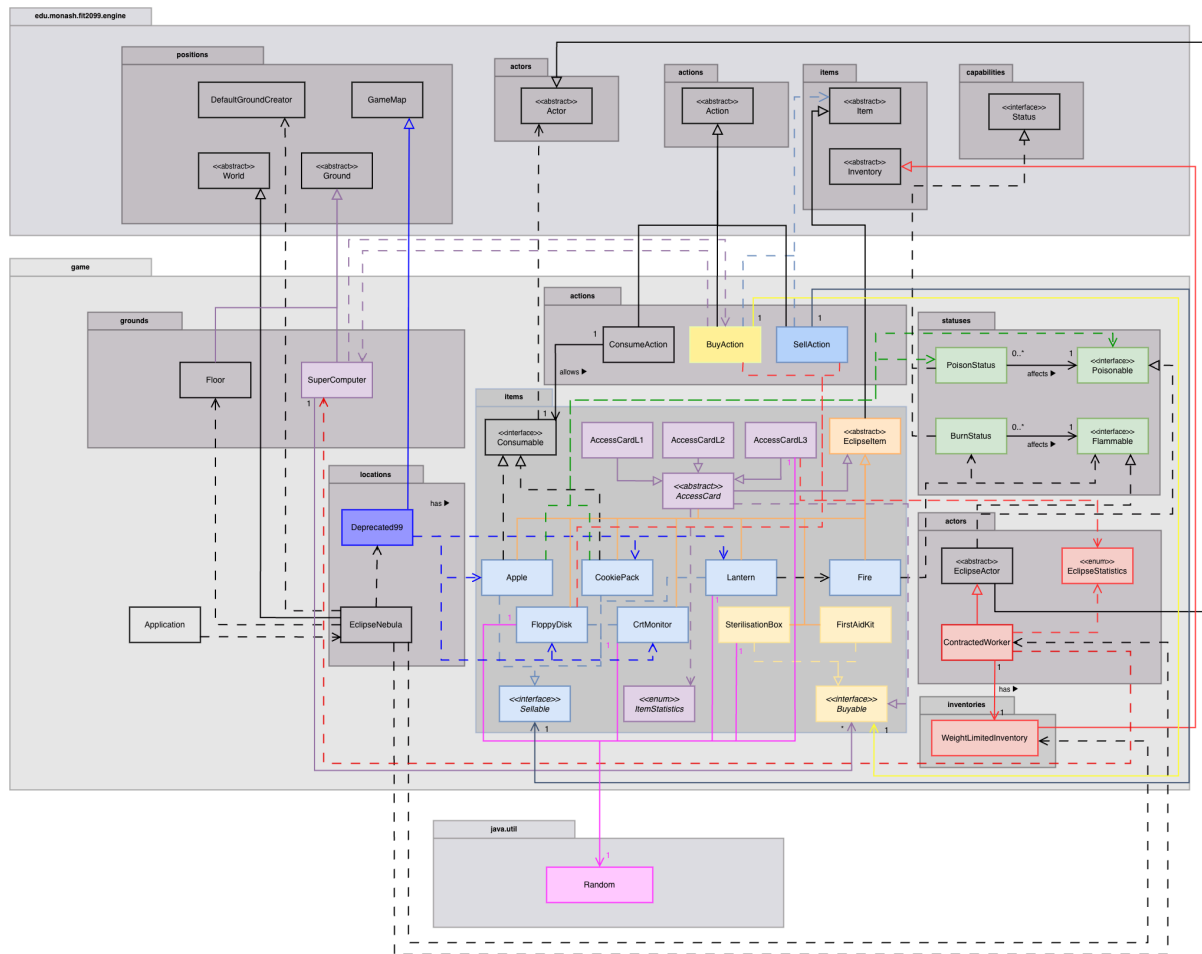
[REQ 1]

Name: Erwynna Soo Wen Xin

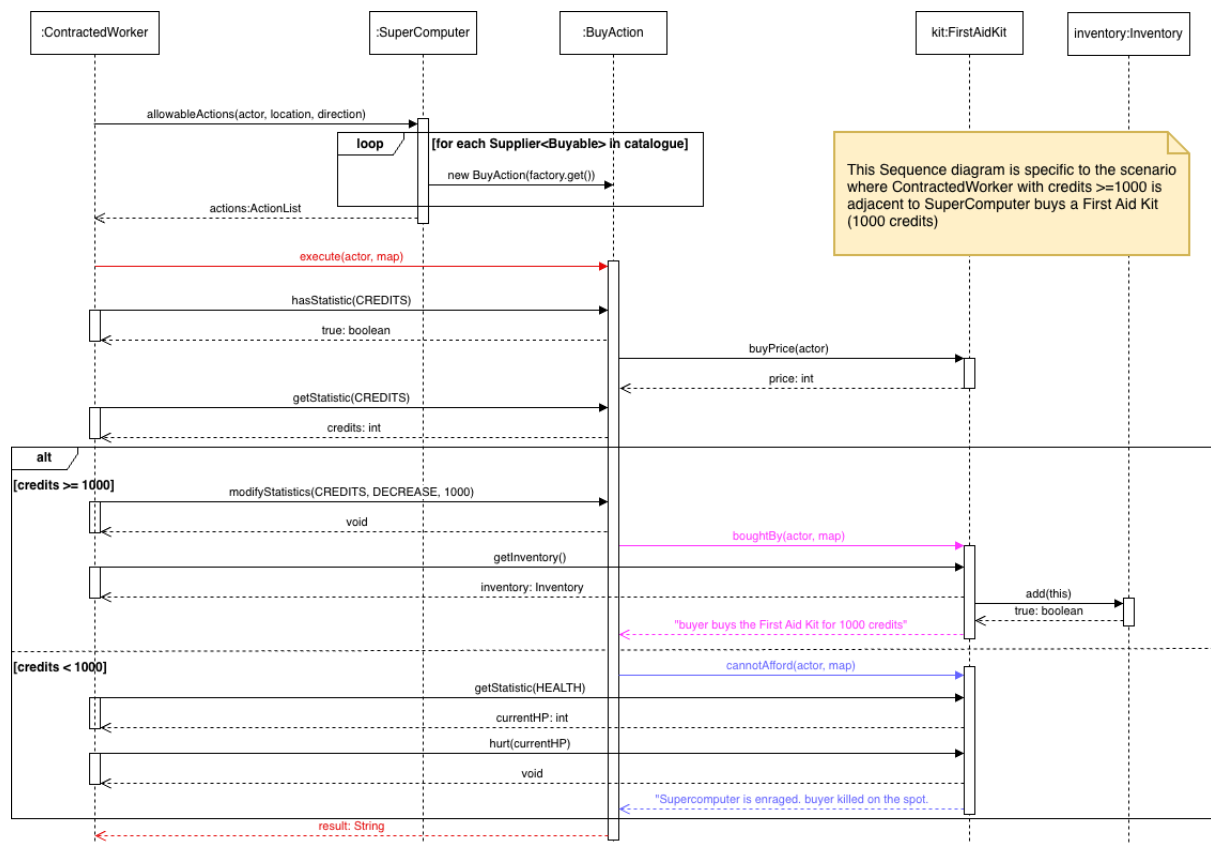
Student ID: 36555789

Design Diagrams

UML



Sequence Diagram



REQ 1

A. SuperComputer Catalogue and the Buy/Sell Pipeline

Design 1: Catalogue Injected as a List of Suppliers (Implemented Design)

In this design, the SuperComputer holds its catalogue as a `List<Supplier<Buyable>>` rather than a list of fixed item instances. Each entry in the list is a small factory that produces a fresh Buyable when called. The constructor takes this list as a parameter, and a static `defaultCatalogue()` method on SuperComputer provides the standard line-up of items that EclipseNebula uses when registering the terminal on the map.

When the engine asks the terminal for actions through `allowableActions()`, the SuperComputer iterates over the list, calls each supplier to create a new Buyable, and wraps it in a BuyAction. This means the terminal never holds the items themselves; it only holds the recipes for making them. As a result, every purchase produces a brand new instance, which avoids the situation where two actors would otherwise share the same item reference.

The catalogue is also typed against the Buyable interface rather than any specific item class. Anything that implements Buyable can be added to the catalogue without changing the SuperComputer. This keeps the terminal completely unaware of how individual items behave or what side-effects they trigger, and lets new items join the catalogue by being added to the static factory method rather than by editing the terminal itself.

PROS & CONS of Design 1

Pros	Cons
<u>Decouples the terminal from concrete items</u> , so it depends only on the Buyable abstraction and can stock anything that implements it.	<u>Adds a small layer of indirection</u> , since each catalogue entry is a factory call rather than a direct field.
<u>Produces fresh instances on every purchase</u> , preventing accidental sharing of a single item reference between actors.	<u>Requires the static factory method to be kept in sync</u> whenever a new buyable item is introduced.
<u>Supports the Open-Closed Principle</u> because new items extend the catalogue without modifying SuperComputer.	<u>Uses generic types</u> (<u>Supplier<Buyable></u>), which may be slightly less familiar to read than a plain item list.

Design 2: Hard-Coded Catalogue Inside the SuperComputer (Alternative Design)

In this design, the SuperComputer would internally create its own item instances inside allowableActions(), constructing each item directly with the new keyword. The catalogue would be a fixed sequence of statements that name every item class explicitly.

Pros	Cons
<u>Straightforward to read</u> because the items the terminal sells are listed inline in one place.	<u>Tightly couples the terminal to every item class</u> , so adding or removing an item forces a change to SuperComputer.java.
<u>Requires no extra types</u> such as Supplier or generic parameters.	<u>Violates the Open-Closed Principle</u> because the terminal must be reopened and edited for every new buyable.
<u>Slightly fewer files to navigate</u> during initial implementation.	<u>Makes testing harder</u> , because the catalogue cannot be swapped out for a smaller test list.

B. Buying and Selling Through Capability Interfaces

Design 1: Buyable and Sellable Capabilities with Generic Actions (Implemented Design)

In this design, the ability to be purchased or sold is treated as a capability rather than an identity. The Buyable interface declares the contract any purchasable item must satisfy (buyPrice and boughtBy), plus an optional cannotAfford hook items can override to react to a broke buyer. The Sellable interface does the same for anything that can be sold (sellPrice, soldBy). Each concrete item implements whichever interface applies to it, so FirstAidKit, SterilisationBox, and AccessCard implement Buyable, while Apple, CookiePack, Lantern, FloppyDisk, and CrtMonitor implement Sellable.

BuyAction and SellAction are written against these interfaces, not against concrete item classes. BuyAction holds a Buyable field, calls buyPrice(actor) to get the cost, and then moves credits by writing directly to the actor's CREDITS statistic (actor.modifyStatistic(EclipseStatistics.CREDITS, DECREASE, price)). Once the price is deducted, BuyAction delegates the rest of the purchase logic to boughtBy(actor, map). SellAction works in mirror image with a Sellable field, sellPrice, the same modifyStatistic call but with INCREASE, and finally soldBy. Because both actions only ever see the interface, a single BuyAction class can purchase every type of buyable item and a single SellAction class can sell every type of sellable item.

Item-specific behaviour, such as the FirstAidKit hurting a broke buyer, the SterilisationBox destroying a random inventory slot, or the Lantern starting nearby fires, lives inside each item's own boughtBy or soldBy method. The action classes never inspect what type of item they are dealing with; the item itself decides what happens. This keeps BuyAction and SellAction free of any item-specific branching.

PROS & CONS of Design 1

Pros	Cons
<u>Eliminates duplicated action classes.</u> since one BuyAction handles every purchasable item and one SellAction handles every sellable item.	<u>Requires every item to implement the correct interface.</u> so introducing a new transactional item is slightly more deliberate.
<u>Encapsulates side-effects inside the item.</u> keeping the action layer thin and free of type checks.	<u>Pushes domain logic into items.</u> which means an item class can grow if its purchase behaviour is complex.
<u>Supports the Dependency Inversion Principle</u> because BuyAction and SellAction depend on abstractions rather than concrete items.	<u>Splits behaviour across two files (action and item).</u> which may need both to be opened when tracing a purchase end to end.

Design 2: One Action Subclass Per Item (Alternative Design)

In this design, every purchasable item would have its own dedicated action class, such as `BuyFirstAidKitAction` or `BuySterilisationBoxAction`, and the same pattern would be repeated for selling. Each subclass would hard-code which item it deals with and implement that item's side-effects directly inside `execute()`.

Pros	Cons
<u>Each transaction's logic</u> is visible in a single class without needing to follow a delegation chain.	<u>Causes class explosion.</u> with one new action subclass required for every new item.
<u>Items can stay as plain data</u> without implementing any extra interfaces.	<u>Duplicates the same credit-deducting and inventory-adding code</u> across every action subclass.
<u>No generic types or interface plumbing</u> are needed.	<u>Couples actions to concrete item classes,</u> violating the Open-Closed Principle and making the action layer fragile.

C. Item Side-Effects on Sale and Purchase

Design 1: Reusing the Engine Status System Through Capability Interfaces (Implemented Design)

Several items in the catalogue apply effects to the seller or buyer. Selling an Apple poisons the seller unless they are carrying a Sterilisation Box, selling a CookiePack costs the seller one HP per cookie as an organic processing fee, selling a Lantern can burn the seller and ignite adjacent tiles, selling a FloppyDisk has a chance to glitch the terminal into deducting extra credits, and selling a CrtMonitor heals the seller but can short-out and spawn Fires on adjacent tiles. To express these outcomes consistently, each ongoing effect is modelled as a Status that the engine already knows how to tick. PoisonStatus and BurnStatus both implement the engine's Status interface, so they are processed by the engine's existing per-turn machinery rather than by any custom code in the actor.

The actor exposes its eligibility for each effect through small capability interfaces. EclipseActor implements Poisonable and Flammable, which means an item can ask the actor for the corresponding capability through `asCapability(Poisonable.class)` before applying the status. If the actor does not satisfy the capability, the effect is simply not applied, and no exception is thrown. This keeps the item from needing to know what concrete type of actor it is interacting with.

Side-effects that involve the environment, such as the Lantern and CrtMonitor spawning Fire on adjacent tiles, are also expressed in terms of items rather than special cases. Fire is an ordinary EclipseItem that is added to a Location through the engine's existing `addItem` call. Because Status objects manage their own lifetime and Fire is just another item on the map, no part of the actor needs to manually count turns or clean anything up.

PROS & CONS of Design 1

Pros	Cons
<u>Reuses the engine's tick lifecycle</u> , so effects expire automatically without any per-turn bookkeeping inside the actor.	<u>Adds extra interfaces (Poisonable, Flammable)</u> , which the actor must implement to be a valid target.
<u>Decouples items from concrete actor types</u> by routing every effect through a capability lookup rather than a class check.	<u>Requires an effect-application path</u> for each new status, since the actor must declare the matching capability.
<u>Keeps environmental effects (fire on adjacent tiles)</u> consistent with normal item placement, avoiding a separate effects subsystem.	<u>Means side-effects are spread across several small classes</u> rather than living in one place.

Design 2: Custom Per-Effect Classes Managed by the Actor (Alternative Design)

In this design, each effect would be its own dedicated class (such as `PoisonEffect` and `BurnEffect`) and would be held in a collection on the actor. The actor would iterate over this collection each turn and call an `apply` method on every effect to reduce duration and apply damage.

Pros	Cons
<u>Behaviour for each effect</u> lives in one self-contained class.	<u>Forces the actor to manage effect lifecycles manually</u> , duplicating logic the engine already provides through <code>Status</code> .
<u>Avoids the engine's status system</u> , which may feel more lightweight at first.	<u>Bypasses an engine feature</u> that exists precisely for this purpose, reducing consistency with the rest of the codebase.
<u>Effects are easy to find</u> because they all sit in the actor's effect list.	<u>Makes the actor responsible</u> for every effect type, so adding new effects forces the actor class to grow.

D. Access Card Hierarchy

Design 1: Abstract AccessCard with Tier-Specific Subclasses (Implemented Design)

Access cards come in three clearance levels, each with the same overall purpose but with different costs and different side-effects on purchase. To capture this shared structure, AccessCard is modelled as an abstract class that implements Buyable and stores the clearance tier as an ItemStatistics.CLEARANCE_LEVEL statistic on the item itself. The three concrete subclasses, AccessCardL1, AccessCardL2, and AccessCardL3, each pass their own tier value (1, 2, or 3) into the AccessCard constructor and override boughtBy to encode the behaviour that makes them distinct.

AccessCardL2 hurts the buyer for a fixed blood-sample amount as part of boughtBy, while AccessCardL3 has a chance to charge a hidden fee on top of the listed price by writing directly to the buyer's EclipseStatistics.CREDITS statistic. AccessCardL1 has no side-effect and just lands in the inventory. The clearance tier is exposed to the rest of the game through a single getter, getClearanceLevel(), which simply reads the underlying statistic. This means REQ2 doors can ask "how strong is this card?" without ever inspecting the card's runtime class.

This arrangement keeps each card's individual logic in its own subclass while sharing everything that is genuinely common (the cost, the symbol, the clearance statistic, the Buyable contract, the inheritance from EclipseItem) in the abstract parent. Adding a future card tier would mean adding one new subclass and choosing one new integer for the statistic, with no change required to the SuperComputer or to BuyAction.

PROS & CONS of Design 1

Pros	Cons
<u>Shares common state and behaviour</u> (price, weight, symbol, clearance statistic) through inheritance, avoiding duplication across the three card classes.	<u>Introduces a small class hierarchy</u> for what is conceptually a single concept (a card).
<u>Encapsulates each tier's quirks</u> inside its own subclass, keeping behaviour local and easy to find.	<u>Requires a new subclass</u> every time a new clearance tier is introduced.
<u>Exposes clearance level through a single getClearanceLevel() getter</u> backed by an ItemStatistics value, so other systems can read a card's tier without inspecting its runtime type.	<u>The 'clearance level' is now an integer statistic</u> rather than a dedicated enum, which makes it slightly less type-safe than the old approach.

Design 2: Single AccessCard Class Parameterised by Level (Alternative Design)

In this design, there would be just one concrete AccessCard class, and the level would be passed into the constructor as an integer. All tier-specific behaviour, such as the L2 blood sample and the L3 hidden fee, would live inside a single boughtBy method that branches on the level value.

Pros	Cons
<u>Fewer classes to maintain</u> , with all card logic centralised in one file.	<u>Centralises behaviour</u> through level branching, which grows messier as new tiers are added.
<u>Easier initial setup</u> because there is no hierarchy to design.	<u>Violates the Open-Closed Principle</u> since a new tier forces a change inside the existing class rather than an extension.
<u>Constructor-driven</u> , so creating any tier is a single new statement.	<u>Mixes concerns inside one class</u> , weakening cohesion and making each tier's behaviour harder to locate.

Conclusion of REQ1

The implemented design treats every part of the SuperComputer transaction as a composition of small, abstraction-driven pieces rather than as one large branching system. The terminal depends only on a list of Buyable suppliers, the action layer depends only on the Buyable and Sellable interfaces, the items themselves carry their own side-effect logic, and the access card tiers are organised through a focused inheritance hierarchy. None of these pieces needs to know the concrete type of the other.

Together, this gives the system a clear separation of responsibilities. The SuperComputer is in charge of presenting a catalogue; the actions are in charge of orchestrating credit movement through the statistical system; the items are in charge of what happens when a transaction succeeds; and the engine status system is in charge of the lifetime of every effect. Each part can be reasoned about on its own, and the points where they meet are narrow interfaces rather than concrete classes.

The design applies the Open-Closed Principle by allowing new buyable or sellable items to be added without modifying the action layer or the terminal; the Dependency Inversion Principle by implementing actions through interfaces; and the Single Responsibility Principle by keeping each class focused on a single concern. The trade-off is a slightly larger number of small types, but this is justified by the way the system stays clean as items and effects multiply across later requirements.

The alternative designs are shorter at first glance, but each one concentrates logic in a place that would have to be reopened and edited every time a new item, effect, or tier is introduced. The implemented design absorbs that growth through extension instead, which is the behaviour expected of an engine-based codebase of this size.